Pure Fabrication

Pure Fabrication Principle

Invent a class to keep design clean

DEFINITION

Assign a cohesive responsibility to an artificial class (not a domain concept) to protect High Cohesion and Low Coupling.

Sometimes no domain concept is the right home for a responsibility — putting it on one would hurt cohesion or coupling. Pure Fabrication invents a class that doesn't represent anything in the problem domain (a Repository, Service, Mapper) purely to give that responsibility a clean, cohesive home. It is a deliberate, design-driven abstraction rather than a model of reality.

WHY IT MATTERS

Information Expert would push persistence onto the entity that owns the data — but database I/O has nothing to do with domain rules, so it bloats the entity and couples it to infrastructure. A fabricated class absorbs that duty, keeping the domain pure and reusable. The price is an extra, non-obvious class, so fabricate only when honouring Expert would actually hurt.

Vasyl Krupka · Ukraine

— How it works

Fabrication	An invented, non-domain class created to hold a responsibility cleanly (Service, Repository, Mapper).
Reason	Protecting High Cohesion and Low Coupling when no domain object is a good home.
Examples	Repository (persistence), Service (coordination), Mapper / Adapter (translation).

HOW TO APPLY

1. Notice a responsibility that would bloat a domain class or couple it to infrastructure.
2. Confirm no existing domain concept is a cohesive home for it.
3. Invent a class named for the responsibility, not for a domain noun.
4. Move the duty there; keep the domain entity focused on its own rules.

LITMUS TEST

Is a responsibility forcing a domain class to do unrelated work (I/O, mapping, coordination)? Would placing it on an entity hurt cohesion? Then fabricate a purpose-built class.

— Smell → fix

Information Expert says Order owns its data, so persistence lands on Order — now the entity knows SQL:

```
class Order {
items: Item[] = []
save() { this.db.query('INSERT ...') } // tied to the DB
}
```

↓ FABRICATE A HOME FOR IT

```
class Order { items: Item[] = [] } // pure domain, no I/O
class OrderRepository { // pure fabrication
save(o: Order) { this.db.query('INSERT ...') }
}
```

Order stays a clean domain concept; OrderRepository is a cohesive, invented home for persistence. The cost is more classes — fabricate only when Expert would hurt.

USE WHEN

- ✓ Persistence / mapping / coordination off a domain entity
- ✓ Expert would bloat the data-holder

AVOID WHEN

- ▲ Inventing classes with no clear responsibility

Vasyl Krupka · Ukraine

— Trade-offs & pitfalls

PROS

+ Keeps the domain clean
+ Reusable, cohesive helpers

CONS

− Too many fabrications fragment the design

COMMON PITFALLS

▲ Over-fabricating until the domain is anemic — data in entities, all behaviour in services.
▲ Vague "Manager / Helper / Util" fabrications with no single, nameable responsibility.
▲ Fabricating before Information Expert actually fails — adding a class with no real payoff.

WHERE YOU SEE IT

- OrderRepository keeping SQL out of the Order entity.
- A PriceCalculator or TransferService coordinating across several aggregates.
- A Mapper / Assembler translating between domain models and DTOs.

SEE ALSO

Information Expert	the principle Pure Fabrication overrides when Expert would hurt cohesion
Repository	the canonical fabrication — persistence lifted off the domain entity
High Cohesion	the property Pure Fabrication protects
Low Coupling	keeps the domain from depending on infrastructure

— Interview & sources

When does Pure Fabrication beat Information Expert?

When Expert would force unrelated duties (DB I/O, mapping) onto a domain class and wreck cohesion — fabricate a Service or Repository instead.

Is a Service class always a Pure Fabrication?

Often yes — Service, Repository, Mapper, Controller are classic fabrications. The test: they exist for software-design reasons, not because they model a domain noun.

Risk of overusing it?

A domain drained into managers and services becomes anemic — data in entities, all behaviour in fabrications. Fabricate to protect cohesion, not as a default.

Doesn't Pure Fabrication contradict object-oriented modelling?

It bends "model the real world" on purpose: not every responsibility maps to a domain noun. Fabrications are still cohesive objects — they just exist for software-design reasons (persistence, coordination) rather than to mirror reality.

REMEMBER

When no domain class fits a responsibility, invent one that does.

SOURCES

Craig Larman — "Applying UML and Patterns" (3rd ed., 2004): Pure Fabrication

Eric Evans — "Domain-Driven Design" (2003): Services and Repositories as designed abstractions